

Algorithm: Steady-State Heat Transfer (2D Laplace Equation)

Step 1: Domain and Grid Initialization

- Define the computational domain of size $L_x = 2$ m and $L_y = 1$ m.
- Discretize the domain using a uniform grid size of $\Delta x = \Delta y = 0.01$ m, resulting in $m = 200$ cells in the x-direction and $n = 100$ cells in the y-direction.
- Create a 2D array to store temperature values at each node $T_{i,j}$.

Step 2: Apply Boundary Conditions

- **Dirichlet Boundaries (Left & Right):** Set $T_{0,j} = 1000$ K for the left wall and $T_{m,j} = 500$ K for the right wall.
- **Neumann Boundaries (Top & Bottom Adiabatic):** Utilize ghost cells outside the physical domain to enforce zero temperature gradient ($\frac{\partial T}{\partial y} = 0$).

Step 3: Initial Guess

- Initialize all interior computational nodes to a starting value of 750 K (the average of the two known boundary temperatures) to speed up convergence.

Step 4: Enforce Adiabatic Boundaries

- After before full domain sweep, update the ghost cells of new domain to mirror the adjacent interior nodes of the old domain:
 - Top boundary: $T_{i,n+2} = T_{i,n}$
 - Bottom boundary: $T_{i,0} = T_{i,2}$

Step 5: Iterative Solvers

Execute the selected numerical scheme to solve the discretized Laplace equation (

$$T_{i,j} = \frac{1}{4}(T_{i+1,j} + T_{i-1,j} + T_{i,j+1} + T_{i,j-1})).$$

- **Method A: Jacobi**
 - Read nodal temperatures from the *previous* iteration matrix.
 - Calculate the new temperature for each interior node:
$$T_{i,j}^{new} = 0.25 \times (T_{i-1,j}^{old} + T_{i+1,j}^{old} + T_{i,j-1}^{old} + T_{i,j+1}^{old})$$
- **Method B: Point Gauss-Seidel (PGS)**
 - Read nodal temperatures using the most *recently updated* values within the current sweep.
 - Calculate the new temperature for each interior node:
$$T_{i,j}^{new} = 0.25 \times (T_{i-1,j}^{new} + T_{i+1,j}^{old} + T_{i,j-1}^{new} + T_{i,j+1}^{old})$$

Step 6: Convergence Check

- Calculate the Root Mean Square (RMS) error between the new and old temperature matrices:

$$e = \sqrt{\frac{\sum (T_{i,j}^{new} - T_{i,j}^{old})^2}{(m+1)(n+1)}}$$

- If $e > 10^{-5}$, swap the old and new matrices and repeat from Step 4.
- If $e \leq 10^{-5}$, the solution has converged. Output the final temperature field and extract the centerline data at $y = 0.5$ m.

Test Results

1. Iteration Comparison

Method	Total Iterations
Jacobi Method	32195
Point Gauss-Seidel	22973

We can observe that the Point-Gauss Seidel method gives a solution in fewer iterations than the Jacobi method hence making it a better algorithm.

2. Temperature Contours

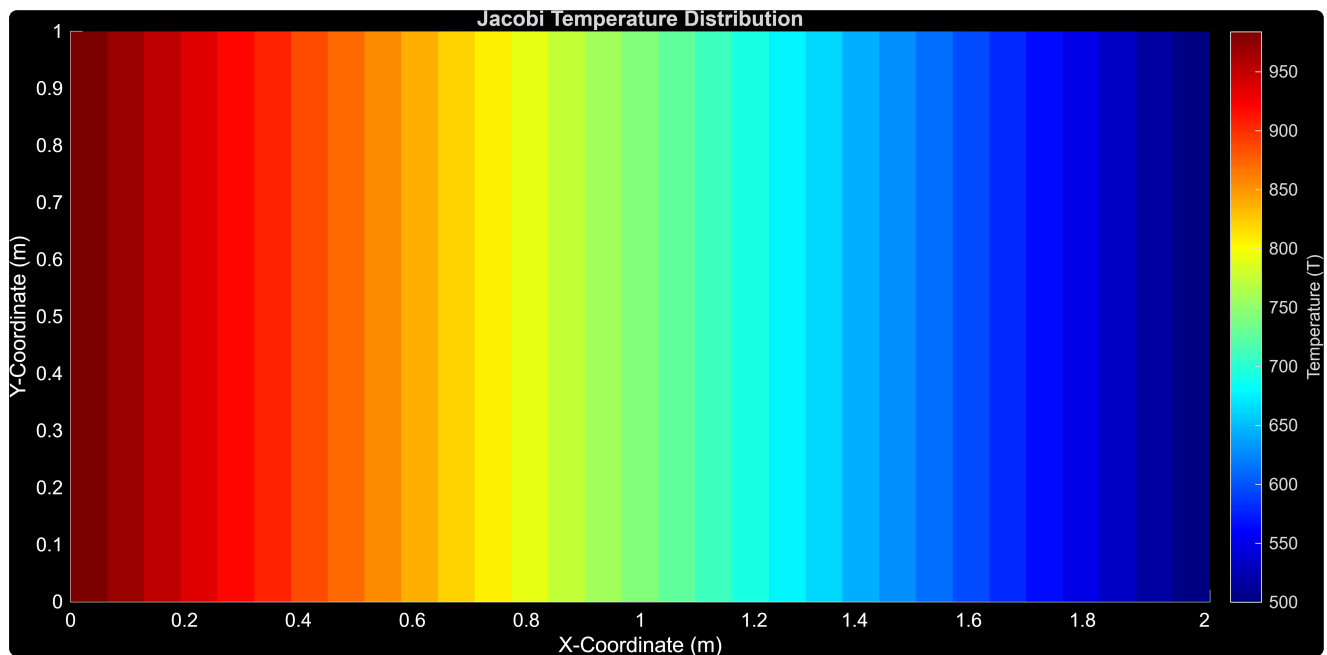


Figure 1: Steady-state temperature distribution using the Jacobi method.

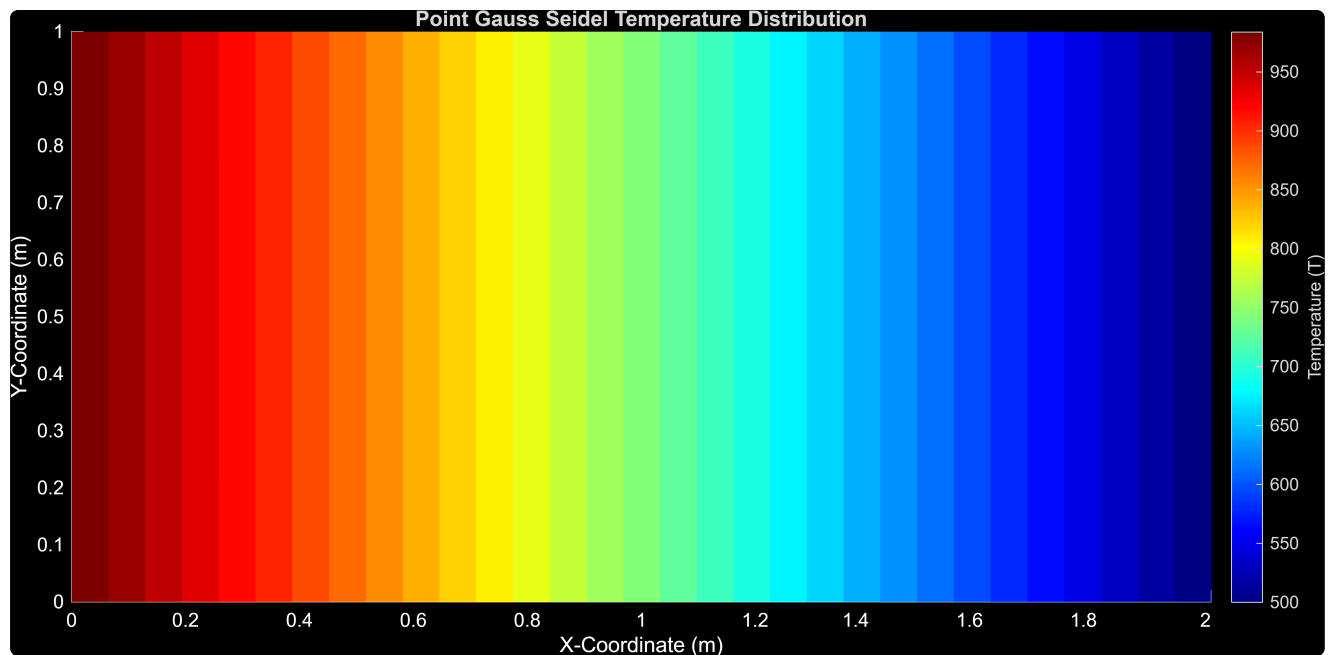


Figure 2: Steady-state temperature distribution using the Point Gauss-Seidel method.

3. Centerline Temperature Distribution ($y = 0.5$ m)

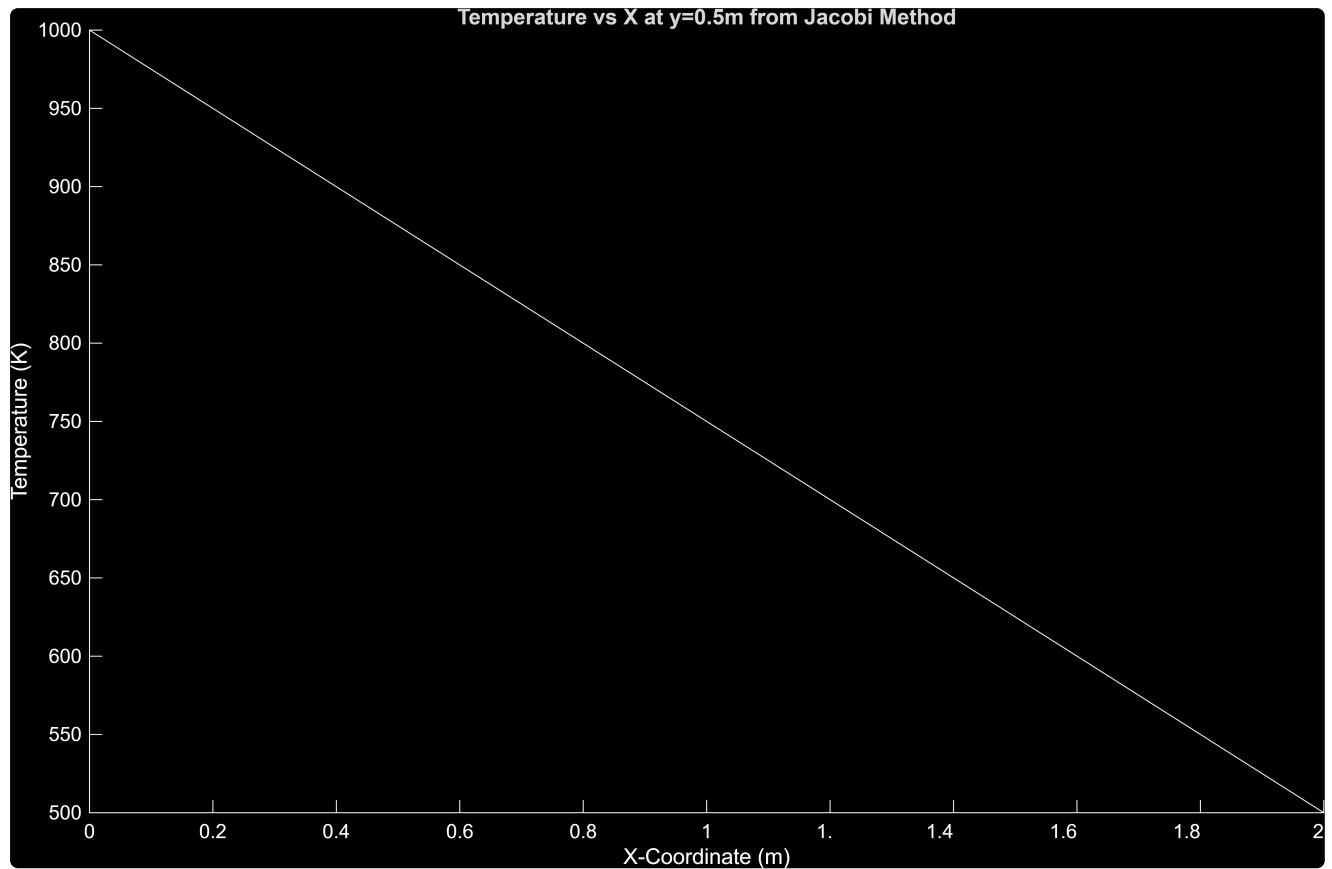


Figure 3: Temperature distribution along the x-axis at $y = 0.5$ m from Jacobi Method

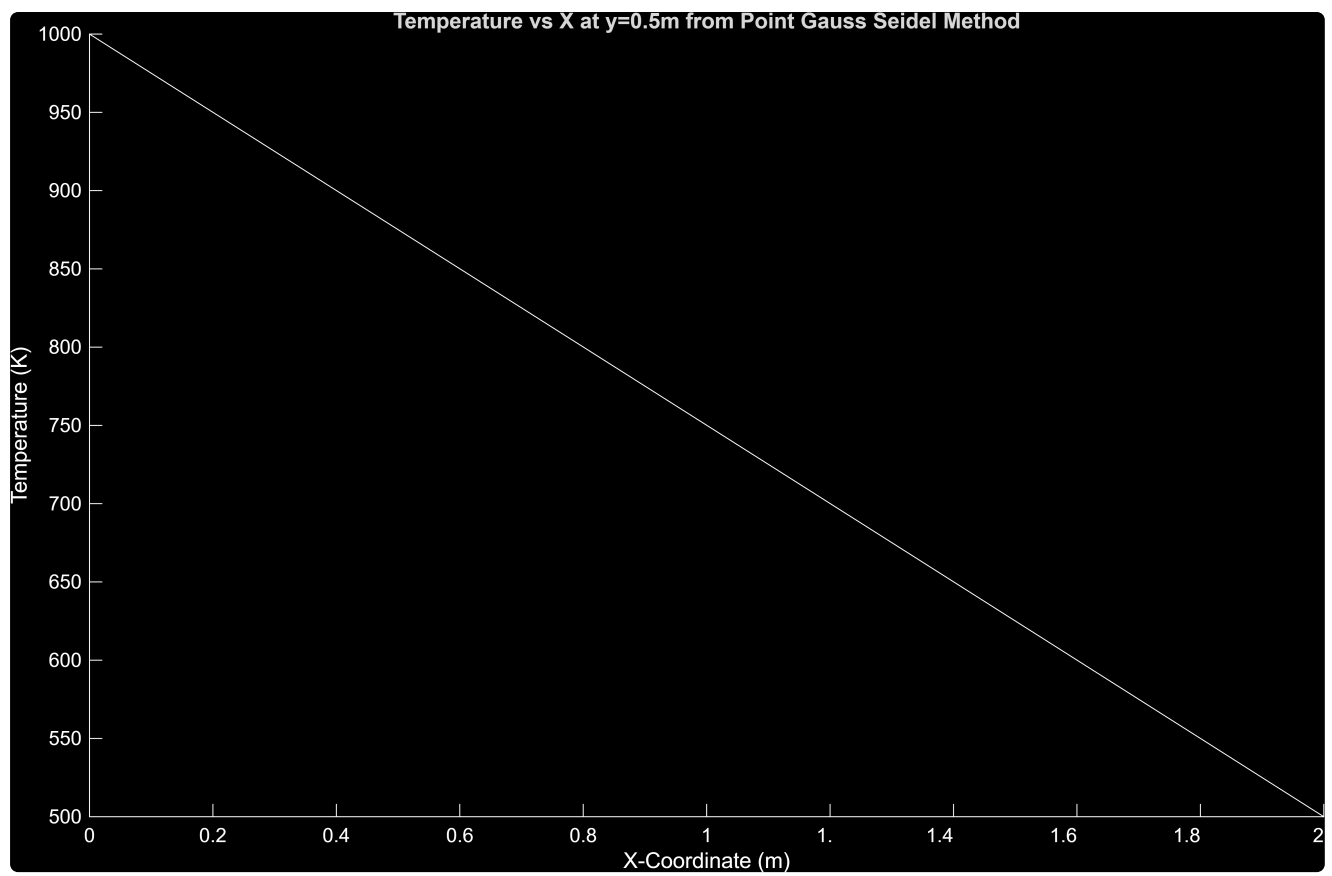


Figure 4: Temperature distribution along the x-axis at $y = 0.5$ m from Point-Gauss Seidel Method